TP2 Team Norms

CSE 360, Fall 2026 (C) · Team 24: Arush Ugal, Karan Patel, Anirudh B, Pranjal Ganvir, Moksha Dev
Revised October 12, 2026, from the TP1 Team Norms of September 15, 2026

How to read this document. This is the TP1 text with every change marked. Blue underlined text was added for TP2; ~~red struck-through text~~ was removed. Each changed norm ends with a **Why changed** line saying what prompted the change. Norms that read No change are word for word as agreed in TP1. Norms 11 and 12 are new.

Norm	Change	Prompted by
1	Meetings set by spacing, not fixed weekdays; summary presenter named in the meeting	TP1 standups
3	Entity classes finished and tested before GUI work starts	TP1 integration
5	Javadoc updated in the same commit as the code	TP1 review
6	AI coaching added on top of validation	TP2 Task 5
8	“Done” requires a positive and a negative test and Javadoc	TP2 Tasks 3, 4, 7
10	Every member checks Canvas after each upload	TP2 instructions
11	New: equal presenting time, two presenters, a named backup	TP1 recording
12	New: pre-submission checklist	TP1 submission

Why we wrote these

Each norm below deals with a specific problem that has caused team projects to fail before. After TP1 we reviewed each one and changed those that TP1 showed were not strong enough. We agreed to all of them together, and each of us agrees to follow them.

Our norms

1. Meet for a standup at least twice a week

We meet ~~Monday and Thursday, with more than 24 hours between meetings.~~ at least twice each week, with at least one full day between two meetings, at a time agreed at the end of the previous standup. Before the meeting ends, one member is named to produce its Status Summary screencast. Each person takes two or three minutes to say what they finished, what they will do next, and what is blocking them.

Why it matters: short, frequent meetings let us catch problems early. If two people are about to change the same class, we find out now instead of during final integration.

Why changed: in TP1 we sometimes ended a standup without deciding who would make the summary video, which delayed the videos. Fixed Monday and Thursday slots also cannot hold the extra standups the TP2 schedule needs in its busiest week; the spacing rule keeps the gap the TP2 instructions require between meetings.

2. Record every standup to the Zoom cloud

We record to the cloud and not to our own computers.

Why it matters: cloud recordings are compressed, so the summary videos stay well under the 100 MB

limit. Local recordings are far too large.

No change.

3. Integrate our work continuously

Nobody saves their work until the end. We merge small working pieces and test the whole application together at least twice a week. A class that other code is built on is finished and passes its tests before that other code is started. For TP2 this means LessonsLearned and LessonsLearnedList come before any lesson window.

Why it matters: splitting up the work, building alone, and combining everything at the end is the most common way these projects fail.

Why changed: TP1 showed that a shared class is safest when it is finished and tested before the code that depends on it is written. TP2 puts three new windows on two new entity classes.

4. Use branches and review all code

Every change goes on its own branch and needs one other person to read it before it is merged.

Why it matters: it stops two people from quietly changing the same method, and it catches mistakes before they reach the shared code.

No change.

5. Keep the documents and the code matching

When the code changes, the design documents and the comments change with it. A method's Javadoc is changed in the same commit as the method, and the reviewer checks it. Screenshots of code are taken again whenever the code they show changes.

Why it matters: if it is not written down, we have to keep explaining it to each other. The documents and the code should look like one person wrote them.

Why changed: in TP1 our comments sometimes lagged behind the code they described. TP2 grades the Javadoc and screenshots of the code directly, so a stale comment or screenshot now costs marks.

6. Check every text input before using it

Every text field has a maximum length. Fields that carry real meaning, such as username, password, and email, get full checks. Lesson fields that benefit from it also get coaching (spelling, completeness, wording) after they pass validation. Coaching advises and never blocks a save. The plan is in AICoaching.pdf.

Why it matters: typing a very long string into a field is a common way to crash a program. If our program crashes, we have shown an attacker a weakness.

Why changed: TP2 Task 5 asks the team to plan for input coaching on top of the validation built in TP1, and our HW2 study of LessonShare showed coaching at entry time improves what gets stored.

7. Answer team messages within 12 hours, and report blockers right away

If something is blocking us, we say so immediately instead of waiting for the next standup.

Why it matters: nobody can help with a problem they do not know about. Waiting a day wastes a day.

No change.

8. Agree on what “done” means

A story is done when it is written, tested, documented, reviewed, and merged. “Tested” means at least one positive and one negative test that pass and are listed in the Requirements to Tests Map. “Documented” means Javadoc on the class, every attribute, and every method, including private ones.

Why it matters: work that is “almost done” hides problems until it is too late to fix them.

Why changed: the TP1 definition did not say what “tested” or “documented” meant. TP2 Task 7 requires positive and negative tests for every requirement, and Tasks 3 and 4 grade the documentation of each attribute.

9. Deal with a teammate who is not helping

We talk to the person directly and early. If nothing changes, we write down what happened and tell the instructional team.

Why it matters: ignoring it rewards the person who is not working and lowers all of the grades.

No change.

10. Upload a safety copy by noon on the due date

We submit whatever we have at noon, then keep working and submit again. After every upload, each member opens the submission in Canvas, downloads it, and posts in the team chat that it opens and is complete.

Why it matters: the last successful submission is the one that is graded. A noon upload gives us hours to fix any problem with the file or with Canvas.

Why changed: the TP2 instructions say every member must verify the submission in Canvas personally, and that this cannot be delegated to one member.

11. Share presenting time equally

Every main screencast has two presenters. Across the three main screencasts and the summary screencasts, each member presents for roughly the same total time. A backup presenter is named before each recording session.

Why changed: (new norm) in TP1 one member was unavailable for a recording, and the member who stood in had not prepared. TP2 Task 9 requires every member to present for roughly equal time.

12. Run a checklist before every upload

Before the ZIP is made, one member who did not build the deliverable checks every file and folder name against the assignment, compiles the project from a clean copy, runs every test suite, deletes all .DS_Store files, and confirms each video is MP4 or MOV and under 100 MB.

Why changed: (new norm) the TP1 upload included 33 stale class files from an old build, and two standup files used the wrong date format in their names. A file over 100 MB would make the whole TP2 submission score zero.

How we handle disagreements

Anyone can raise a problem privately first. If it keeps happening, we discuss it as a team at the next standup. If it still is not fixed, we write it down and take it to the instructional team.

No change.

Signatures

By signing below, each of us agrees to follow these norms as revised for TP2.

Member	Signature	Date
Arush Ugal		
Karan Patel		
Anirudh B		
Pranjal Ganvir		
Moksha Dev		